

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Summary of data sets and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the data set into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

Repo Link:

<https://github.com/Kamlesh-DevOP/deep-learning-lab/tree/main/lab-1>

7. Experimental Results

Dataset Preparation and Preprocessing

For this experiment, we used the UCI Banknote Authentication dataset. It contains 1,372 samples of real and fake banknotes. Each sample has four features that were extracted using a Wavelet Transform: variance (x_1), skewness (x_2), kurtosis (x_3), and entropy (x_4). In total, there are 762 real banknotes (Class 0) and 610 fake ones (Class 1).

To make sure the model trains fairly and no single feature gets learnt a lot compared to others, we prepared the data in two steps:

1. **Stratified Partitioning:** We split the data into 80% for training (1,097 samples) and 20% for testing (275 samples). A stratified split was used to make sure both sets keep the exact ratio of real to fake notes as the whole dataset.
2. **Z-Score Standardization:** All features are standardised to have a mean of zero and a standard deviation of one. This is important because features with naturally larger values (like skewness or kurtosis) could unfairly dominate the error correction process compared to smaller features like entropy.

Training Progress and Observations

We trained the network for 20 epochs using a constant learning rate (η) of 0.01. The weights ($\mathbf{w}$) were initialized with small random numbers from a Gaussian distribution to break symmetry, and the bias (b) was started at 0.0. We updated the model sample-by-sample and noticed three main phases during training:

- **Initial Training Step (Epoch 1):** At first, the random decision boundary made a lot of mistakes, misclassifying 112 samples. After updating the errors, the weights became $\mathbf{w} = [-0.0545, -0.0900, -0.0646, 0.0005]$ with a bias of -0.0300 .
- **Fast Learning Phase (Epochs 2–4):** The model corrected errors very quickly here. The number of misclassified samples dropped to 17 in Epoch 2 and 14 in Epoch 3, reaching a low point of just 7 mistakes in Epoch 4.
- **Bouncing Phase (Epochs 5–20):** Instead of going smoothly down to zero errors, the model's performance bounced around. The mistakes went up to 12 in Epoch 5, hit a high of 29 in Epoch 7, and slowly went down to 9 mistakes by the final epoch. The final weights were $\mathbf{w} = [-0.1591, -0.1865, -0.1628, -0.0173]$ and the bias was $b = -0.0700$.

According to the Perceptron Convergence Theorem, a perceptron will only reach zero training errors if the data is completely linearly separable. Because our model kept bouncing between 7 and 29 mistakes and never reached zero, it proves that the real and fake banknote features overlap slightly. This means the dataset is not perfectly linearly separable in a 4D space.

8. Generated Plots with Interpretation

Exploratory Data Analysis (EDA) Interpretations

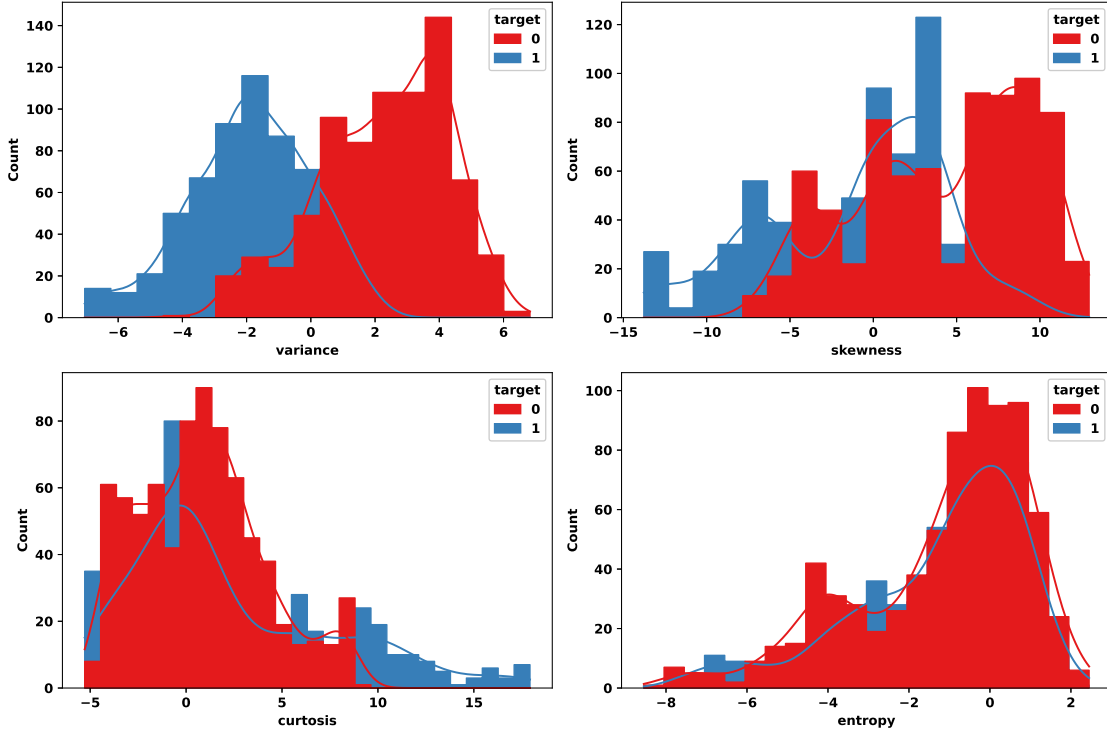


Figure 1: Histograms and Density Estimates of the Banknote Authentication Features grouped by Class.

Looking at Figure 1, the four features show very different patterns:

- **Variance:** This feature separates the classes the best. Real banknotes (Class 0) are mostly on the right side, peaking around 3 to 4. Fake banknotes (Class 1) are pushed to the left, peaking near -2.
- **Skewness:** Real banknotes have two peaks (bimodal) around -5 and 7. Fake banknotes just have one main peak centered around 3.
- **Kurtosis and Entropy:** Both of these features overlap a lot between the two classes. Entropy looks almost identical for both, peaking near 0. This means neither of these features can separate the classes well on their own.

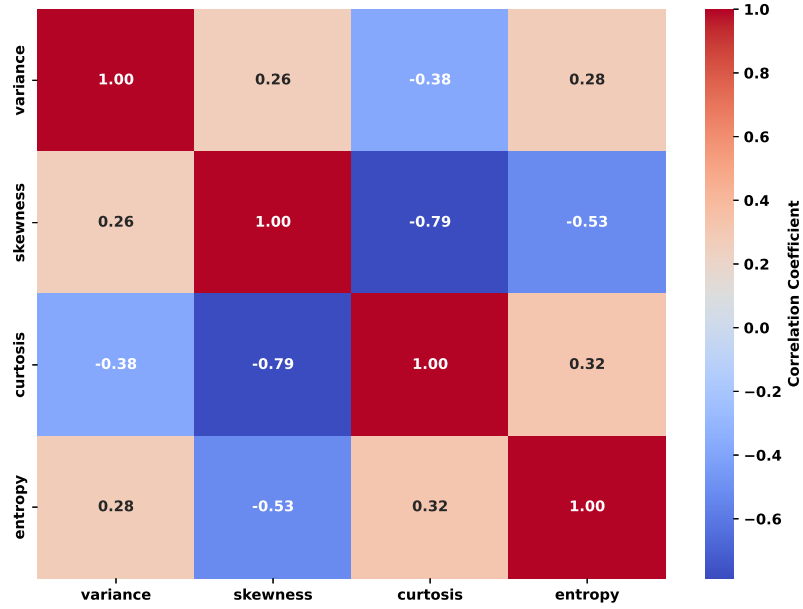


Figure 2: Correlation Matrix Heatmap demonstrating linear relationships between features.

Figure 2 shows the correlation matrix, which tells us how the features relate to each other. The biggest thing we noticed is a strong negative correlation of -0.79 between skewness and kurtosis. This means they kind of give us the same information. On the other hand, variance doesn't have strong correlations with the other features, meaning it gives us unique information.

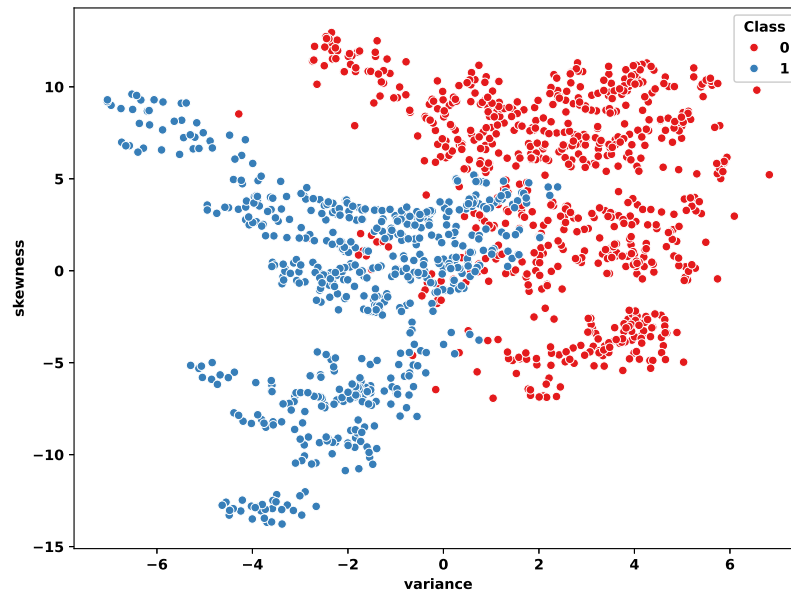


Figure 3: Scatter Plot of Feature 1 (Variance) vs Feature 2 (Skewness) highlighting class separability.

The scatter plot of Variance vs. Skewness in Figure 3 helps us see the classes visually. Real banknotes are mostly in the upper right area, while fake ones are grouped in the lower left (negative variance and lower skewness). We can see a general dividing line, but there is some mixing in the middle. This mixing tells us that a simple straight line won't be able to separate everything perfectly without making some errors.

Training Dynamics and Parameter Evolution

By tracking the model's changes over 20 epochs, we can see exactly how our custom Perceptron learns.

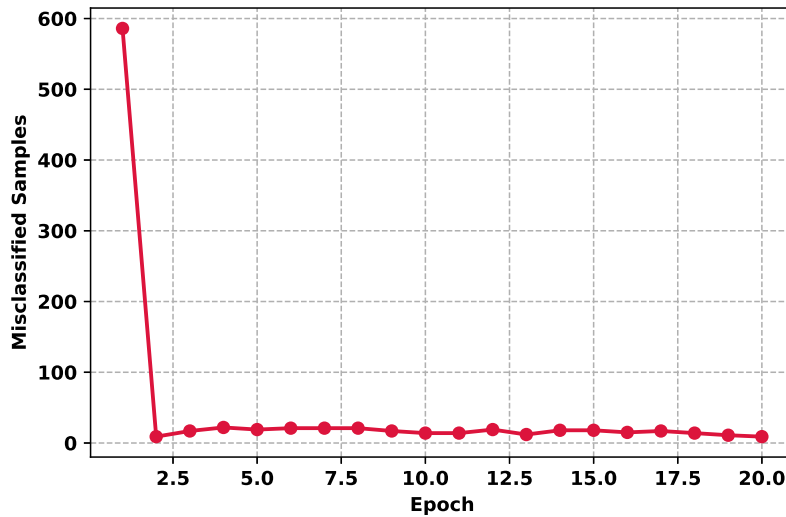


Figure 4: Training Error vs Epoch.

Figure 4 shows the training error. In Epoch 1, the random starting weights caused about 585 misclassifications. By Epoch 2, the model updated itself and dropped the mistakes down to just 17. For the rest of the training, the error just bounced around between 7 and 29. This bouncing happens because of the Step activation function. Since it uses a hard threshold instead of smooth updates, the decision line keeps jumping back and forth when it hits overlapping data points.

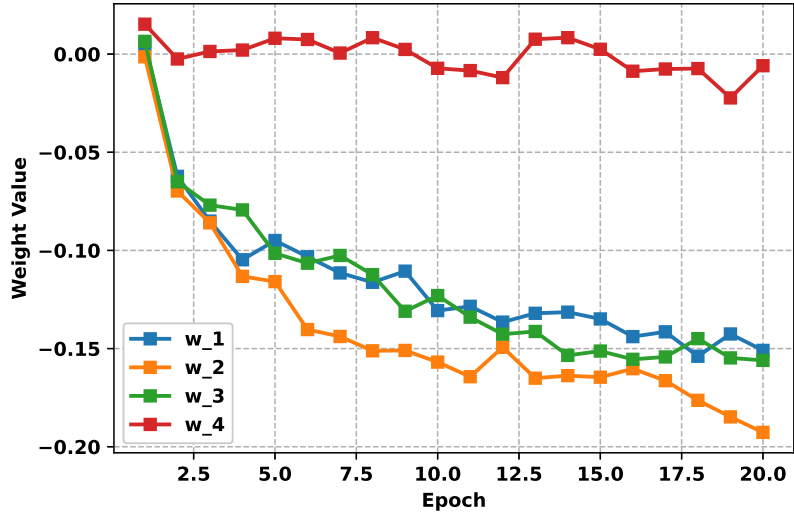


Figure 5: Weight Evolution.

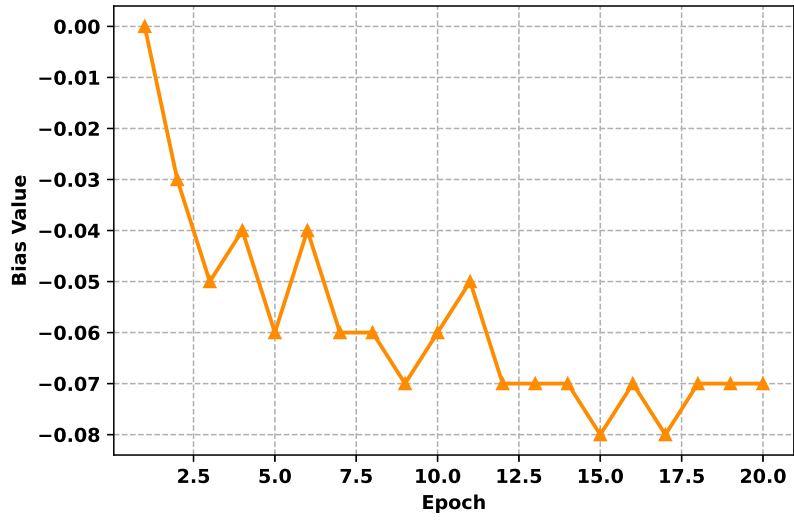


Figure 6: Bias Evolution.

Figures 5 and 6 show how the weights and bias changed over time:

- **Weights (w_1, w_2, w_3):** The weights for variance (w_1), skewness (w_2), and curtosis (w_3) all dropped from 0.0 into negative numbers. Weight w_2 dropped the most, ending near -0.19, while w_1 and w_3 settled around -0.15 to -0.16. The model made these weights negative to penalize fake banknotes that have high values in these features.
- **Weight (w_4):** The weight for entropy (w_4) stayed right around 0.0 the whole time. This matches what we saw in the EDA: entropy doesn't really help the model draw a line between the two classes.
- **Bias (b):** The bias dropped right away from 0.0 to -0.03, then oscillated between -0.04 and -0.08 before ending at -0.07. This bouncing shows the model trying to shift the decision line back and forth to find the best balance between the classes.

Model Evaluation and Comparative Analyses

Finally, we checked how well the model works on unseen test data.

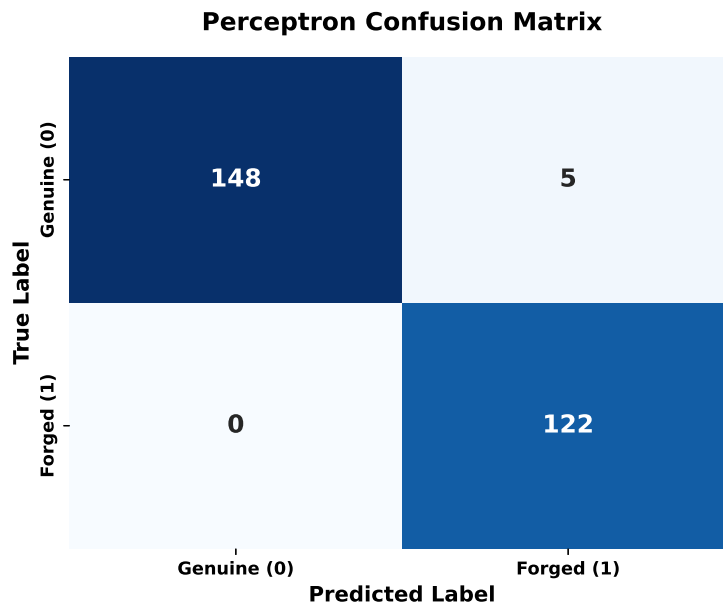


Figure 7: Perceptron Confusion Matrix.

The confusion matrix in Figure 7 shows the model did a great job. Out of 275 test samples, it correctly found 148 real notes and 122 fake ones. The best part is that it had 0 False Negatives, meaning no fake banknotes slipped through as real. The only mistakes were 5 False Positives, where it thought a real note was fake.

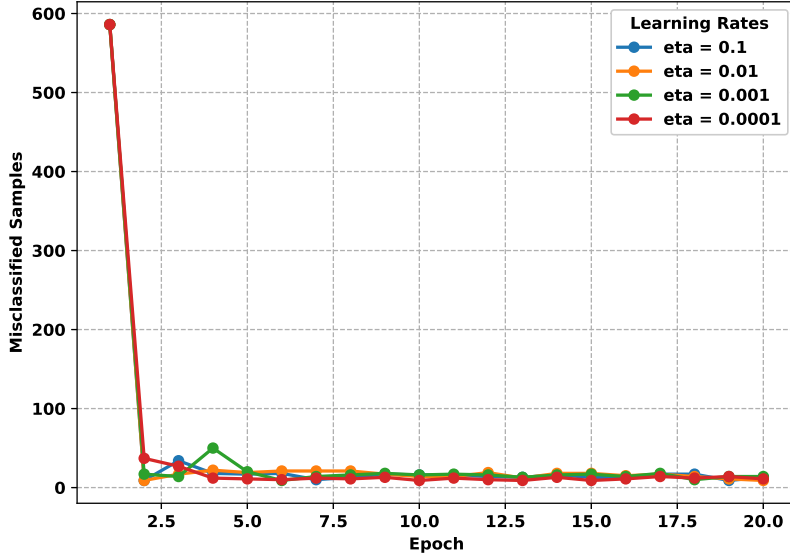


Figure 8: Learning Rate Comparison.

The above figure compares the model using four different learning rates ($\eta = 0.1, 0.01, 0.001$, and 0.0001). All of them dropped the error from 585 down to a low number in just two epochs. There were some small differences—like a quick spike to 50 errors for $\eta = 0.001$ around Epoch 4—but all of them ended up bouncing in the same error range after Epoch 5. This tells us that the remaining mistakes are because the data isn't perfectly separable with a straight line, not because we picked the wrong learning rate.

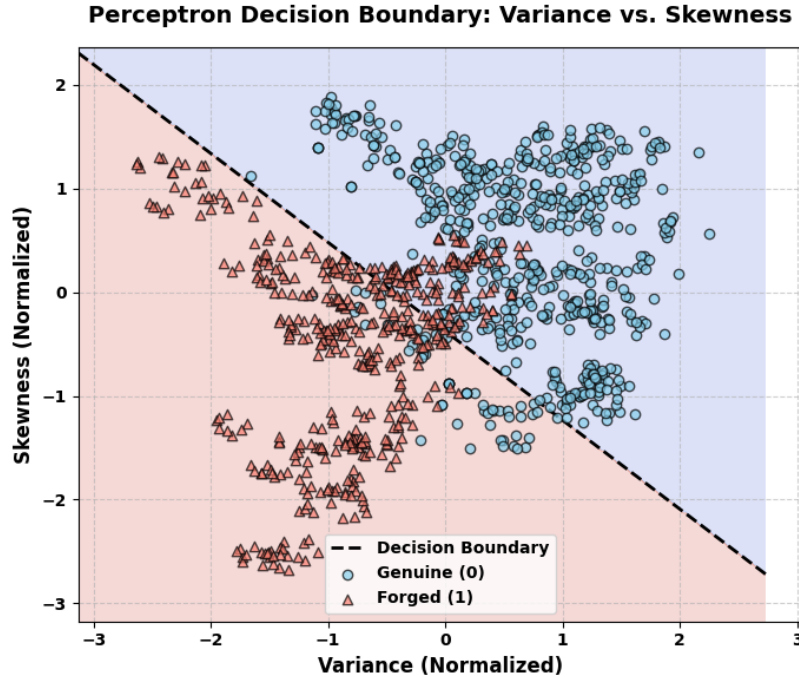


Figure 9: Variance vs Skewness Boundary.

Figure 9 shows the final decision line drawn on a plot of variance vs. skewness. The perceptron drew a straight diagonal line. The blue area on the top right is where it predicts real notes, and the red area on the bottom left is for fake notes. Red triangles and blue circles are seen mixing right on top of the dashed boundary line. This mixing is the visual proof of why our single-layer model can't get zero errors—a straight line just can't separate those overlapping points perfectly.

9. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80% (1097) / 20% (275)
Learning Rate	0.01
Epochs	20
Final Weights	$[-0.1591, -0.1865, -0.1628, -0.0173]$
Final Bias	-0.0700
Accuracy	98.18%
Precision	0.9606
Recall	1.0000
F1-score	0.9799

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	112	-0.0545	-0.0900	-0.0646	0.0005	-0.0300
2	17	-0.0820	-0.1008	-0.0794	-0.0066	-0.0400
3	14	-0.0777	-0.1150	-0.0942	-0.0005	-0.0400
4	7	-0.0893	-0.1176	-0.0950	0.0052	-0.0500
5	12	-0.0969	-0.1156	-0.1109	-0.0039	-0.0500
6	16	-0.1015	-0.1268	-0.1279	0.0097	-0.0500
7	29	-0.1013	-0.1367	-0.1260	0.0078	-0.0600
8	22	-0.1128	-0.1401	-0.1284	-0.0021	-0.0600
9	15	-0.1274	-0.1487	-0.1126	-0.0004	-0.0600
10	17	-0.1348	-0.1529	-0.1207	-0.0004	-0.0600
11	15	-0.1297	-0.1605	-0.1270	0.0052	-0.0600
12	12	-0.1289	-0.1526	-0.1411	-0.0049	-0.0800
13	16	-0.1272	-0.1608	-0.1485	-0.0070	-0.0700
14	15	-0.1384	-0.1668	-0.1405	-0.0213	-0.0600
15	11	-0.1454	-0.1737	-0.1445	-0.0024	-0.0600
16	9	-0.1356	-0.1801	-0.1462	-0.0032	-0.0800
17	14	-0.1556	-0.1810	-0.1423	-0.0082	-0.0700
18	14	-0.1499	-0.1911	-0.1458	-0.0189	-0.0700
19	13	-0.1505	-0.1932	-0.1578	-0.0056	-0.0700
20	9	-0.1591	-0.1865	-0.1628	-0.0173	-0.0700

10. Discussion

Step vs. Sigmoid Activation Functions

The traditional Perceptron utilizes a hard Step Activation function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases} \quad (1)$$

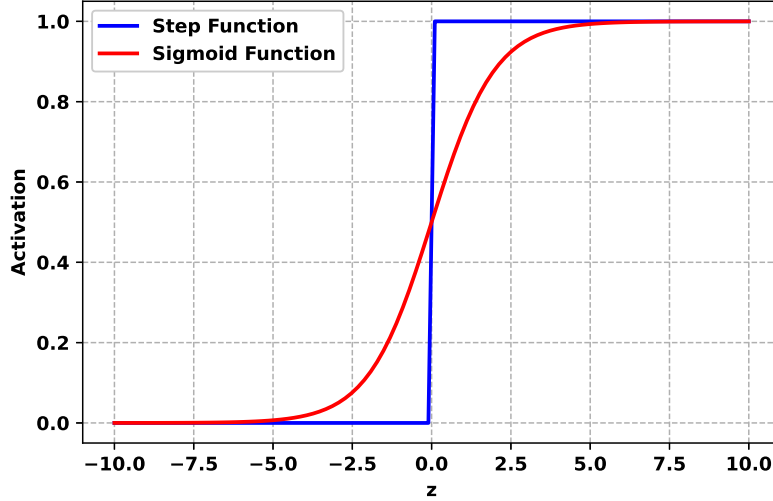


Figure 10: Activation function Comparison.

While computationally simple, the Step function is not used in modern deep learning models because it is non-differentiable at $z = 0$ and its derivative is exactly zero everywhere else. Modern neural networks rely on gradient descent and the backpropagation algorithm, which require continuous, differentiable activation functions to calculate weight update gradients via the chain rule.

The Sigmoid function, defined as $f(z) = \frac{1}{1+e^{-z}}$, provides a smooth, bounded, and differentiable curve that outputs a continuous probability score between 0 and 1. Because its derivative is non-zero, it allows the network to learn progressively from proportional errors rather than binary jumps.

Comparison with Scikit-Learn’s Perceptron

The custom Perceptron implementation was compared against `sklearn.linear_model.Perceptron` using identical hyperparameters ($\eta = 0.01$, 20 epochs).

- **Scikit-Learn Results:** Achieved a test accuracy of 97.45%, with final weights $\mathbf{w} = [-0.1169, -0.1422, -0.1107, 0.0050]$ and bias $b = -0.0300$.
- **Custom Model Results:** Achieved a slightly higher test accuracy of 98.18%, with weights $\mathbf{w} = [-0.1591, -0.1865, -0.1628, -0.0173]$ and bias $b = -0.0700$.

Both models successfully identified that the first three features (variance, skewness, kurtosis) require strong negative scaling, while the fourth feature (entropy) contributes very less for the prediction, since the weights are close to 0. The slight performance variance maybe attributed to Scikit-Learn’s internal default stopping criteria, shuffling mechanisms, and potential regularization penalties that prevent weights from growing as aggressively as they did in the custom, unregularized implementation.

Impact of Varying Learning Rates

The experiment was repeated across three distinct learning rates (η), yielding the following test accuracies:

- $\eta = 0.001$: Accuracy = 98.18%
- $\eta = 0.01$: Accuracy = 98.18%
- $\eta = 0.1$: Accuracy = 98.55%

While all three configurations achieved near-identical classification performance, the training logs revealed distinct geometric scaling. For the large learning rate ($\eta = 0.1$), the weights expanded massively, reaching values near -1.64 . For the smallest learning rate ($\eta = 0.001$), the weight updates were highly conservative, terminating around -0.15 . The identical performance indicates that for this specific dataset, the final angle of the decision boundary was achieved regardless of the absolute magnitude of the weight vectors.

The XOR Problem Limitation

When tested on the non-linear XOR dataset, the model completely failed, predicting an output array of $[1, 0, 0, 0]$ against the true labels of $[0, 1, 1, 0]$. The training log showed the misclassification count permanently stuck at 3 errors, with weight updates ceasing entirely.

This confirms the fundamental mathematical limitation of the Single Layer Perceptron: it can only map linear decision boundaries. The XOR problem requires separating the coordinates $(0, 1)$ and $(1, 0)$ from $(0, 0)$ and $(1, 1)$. It is geometrically impossible to draw a single straight line that isolates these classes. Solving XOR requires a Multi-Layer Perceptron (MLP) capable of transforming the inputs into a higher-dimensional, linearly separable space.

Effect of Feature Normalization on Convergence

To study structural stability, the model was retrained on raw, non-normalized features. The non-normalized model achieved a slightly lower accuracy of 97.82%. More importantly, the parameter scales shifted drastically; the bias increased rapidly to 0.7700, and the weight vectors became heavily skewed (e.g. w_1 reached -0.7729) to compensate for the varying natural scales of the data set. Feature normalization (Z-score scaling) is proven to be critical, as it ensures a symmetric error surface, preventing features with inherently larger absolute numerical ranges from dominating the gradient updates and destabilizing the hyperplane.

11. Additional tasks

Task 1: AND, OR, and NOT Gates

The following figures illustrate the perceptron learning algorithm successfully finding a linear decision boundary for the NOT, AND, and OR logic gates. Since these problems are linearly separable, the perceptron updates its weights for a few steps until it completely separates the target classes (blue circles represent 1, red crosses represent 0).

NOT Gate - Decision Boundary Updates

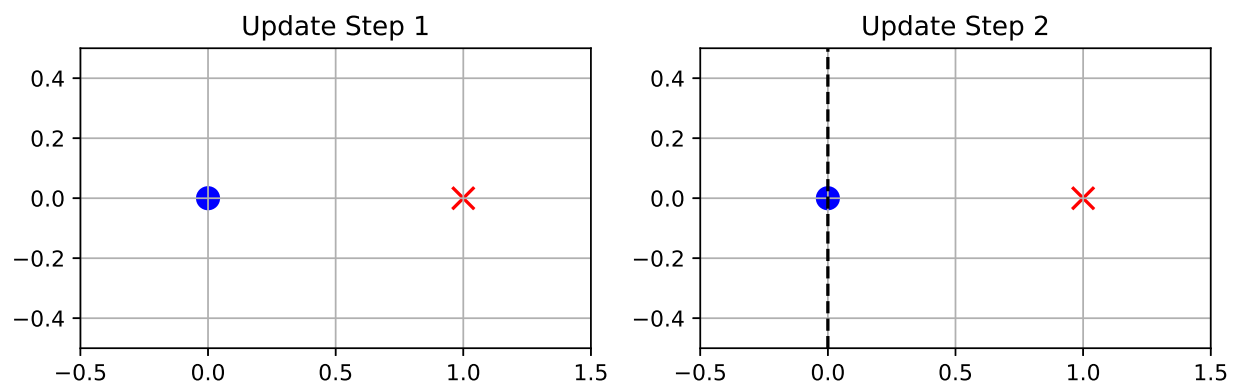


Figure 11: NOT Gate - Decision Boundary Updates

AND Gate - Decision Boundary Updates

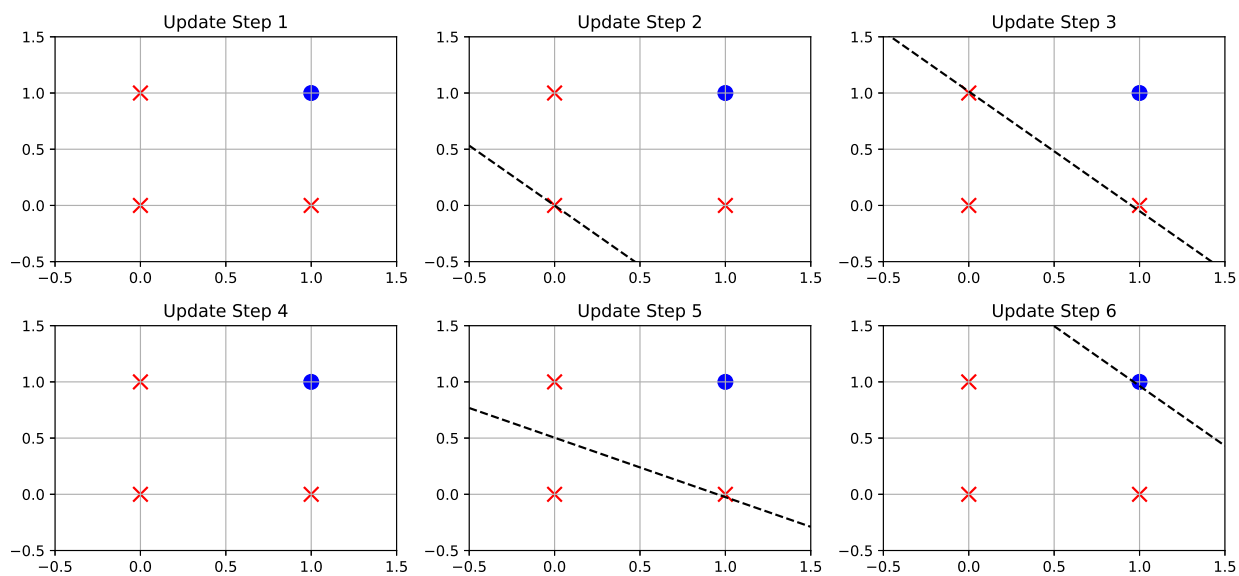


Figure 12: AND Gate - Decision Boundary Updates

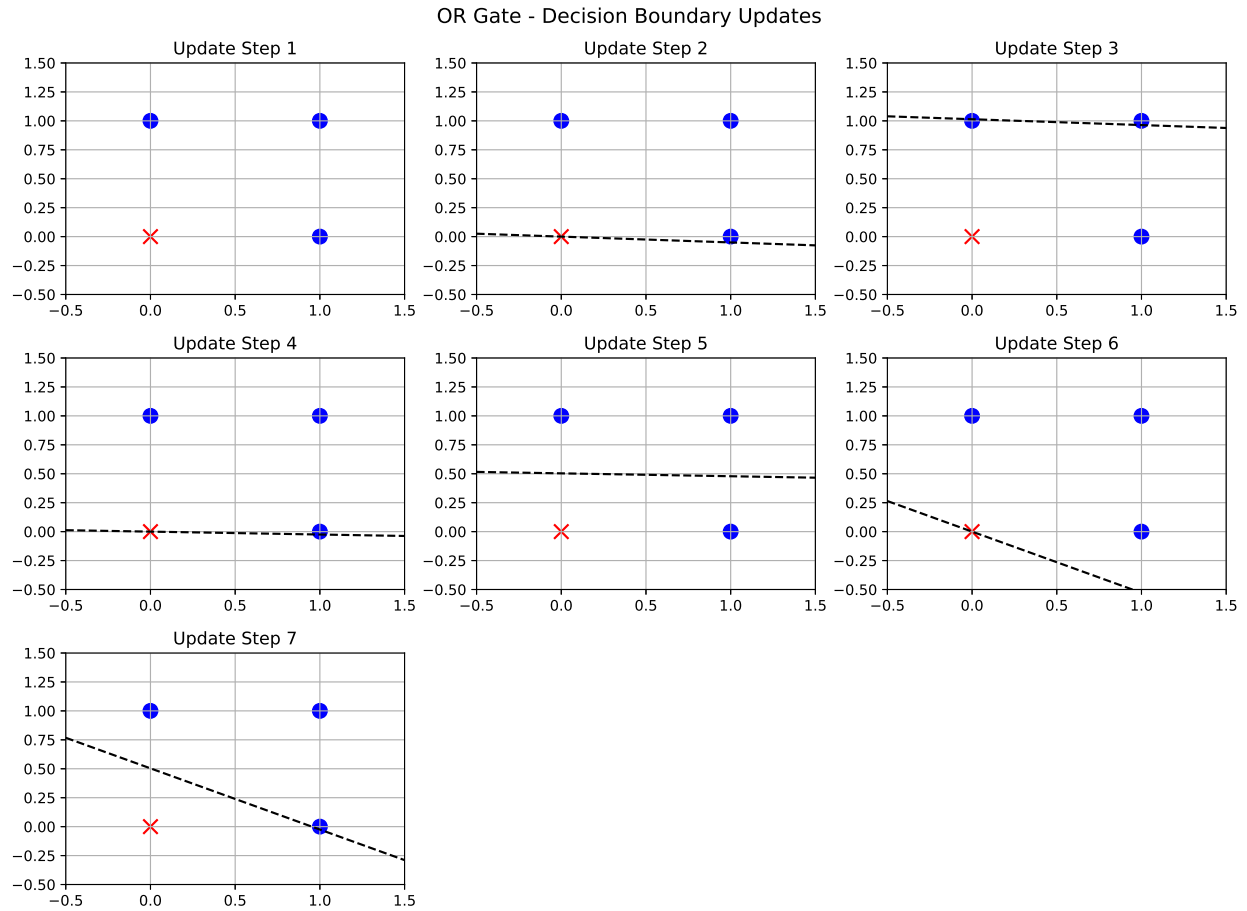


Figure 13: OR Gate - Decision Boundary Updates

Task 2: XOR Gate

Unlike the previous logic gates, the XOR gate is not linearly separable. As shown in the figure below, the perceptron attempts to separate the data, but the linear decision boundary endlessly oscillates. A single-layer perceptron cannot converge on a valid solution for XOR data.

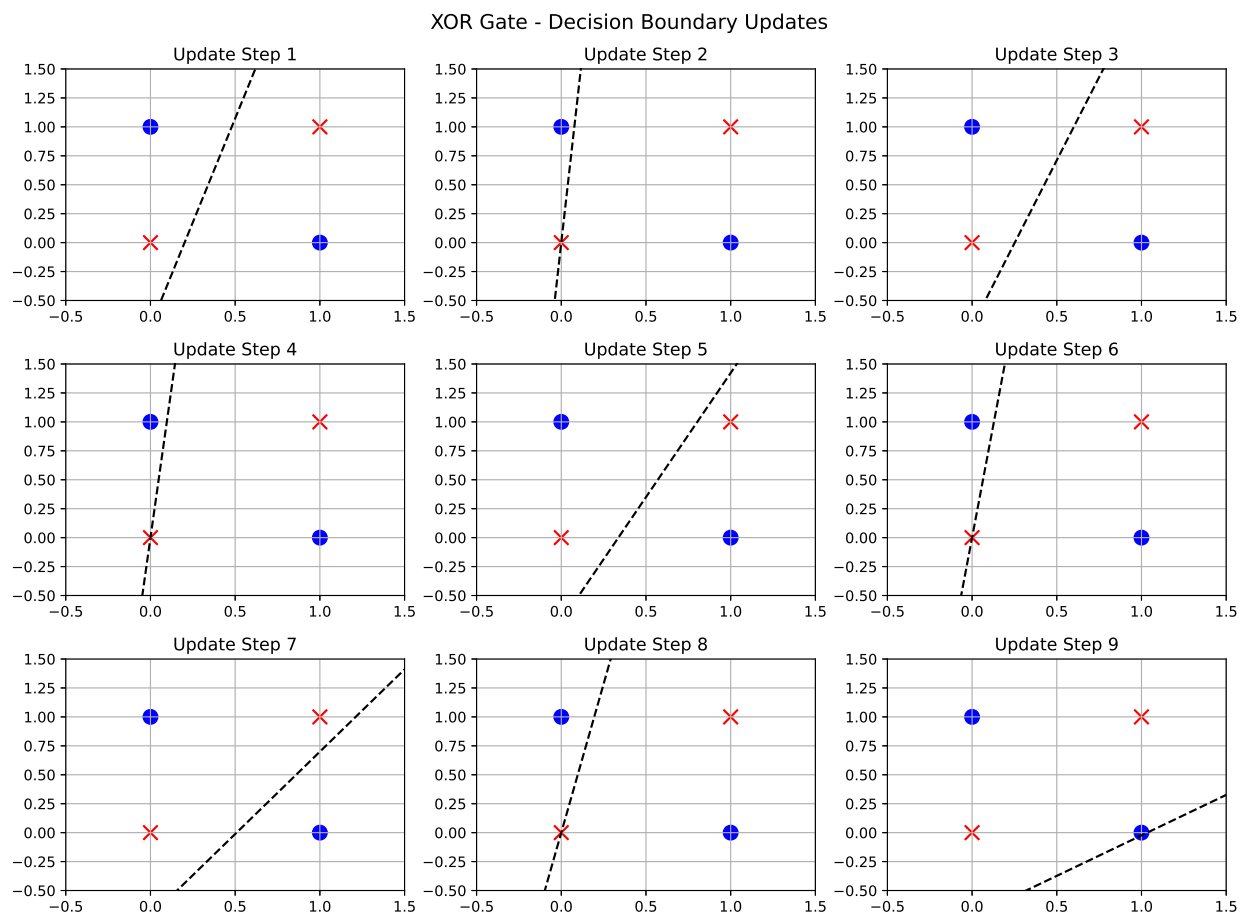


Figure 14: XOR Gate - Decision Boundary Updates

12. Conclusion

In this lab, we learned the basics of artificial neural networks by building a Single Layer Perceptron from scratch. We tested it on the UCI Banknote Authentication dataset, and our custom model got a strong test accuracy of 98.18%. This shows that it can successfully find a good linear decision boundary for real-world data.

During the experiment, we practically verified a few important concepts. From our Exploratory Data Analysis and by tracking the weights, we saw that variance and skewness were strong features for prediction, while entropy didn't help much. We also noticed that the training error kept bouncing around instead of dropping to zero. This bouncing happens because of the Step activation function's hard threshold, and it proves that the dataset has overlapping points and is not perfectly linearly separable.

When we compared our model to Scikit-Learn's Perceptron, we saw that our custom code was mathematically correct since they performed similarly. Testing different learning rates showed us the trade-off between stable training and how large the weights can grow. We also saw the model completely fail on the XOR dataset, which proves a major limitation: a single-layer Perceptron just cannot solve non-linear problems on its own. Finally, our tests showed that standardizing the data with Z-score normalization is extremely important to keep the weight updates stable and balanced

during training.

13. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.